TESTING REPORT: Combat, Pathfinding, Stats, and Entity Systems

Name: Joseph Trieu, Cody Nguyen, Niyousha Aminafshari, Usaid Syed

I. UNIT TESTS OVERVIEW

Combat System Tests: *Test Class: CombatManagerTest (30 tests)*

Features Tested:

- Listener Registration/Removal: Ensures combat events can be observed by multiple listeners
- Damage Notifications: Verifies damage events are propagated to all registered listeners
- Death Notifications: Confirms death events trigger once and are broadcast correctly
- Attack Notifications: Validates attack results (hit, miss, critical) are communicated
- Heal Notifications: Tests healing event propagation
- Concurrent Modifications: Ensures listener list is thread-safe during iteration
- Multiple Listener Handling: Verifies all listeners receive events in order

Tile System Tests: *Test Class: DrawTest (2 tests)*

Features Tested: This class check that the TileManager drawing system works correctly and consistently. They make sure that map tiles and interactive world items are shown on the screen properly, even when what you can see changes.

testDrawVisibleTileRendersPixel(): Ensures a visible tile's image produces at least one red pixel on the render surface.

testDrawComponentIsInvoked(): Validates that adding a component results in its image (blue pixel) appearing on the render surface.

Rationale: Map tiles and world components form the fundamental visible layer of the game.

Test Class: TileManagerTest (18 tests)

Features Tested:

a) **Map Initialization:** verifies map arrays (`mapTileNum`, `mapTiles`) are fully created, checks all tile indices are valid and all tile instances exist, and confirms map dimensions match `GamePanel`'s world size.
b) **Tile Type & Instance Consistency:** ensures tiles inherit attributes from `TileType[]` correctly, collision flags, trap attributes, and coordinates are copied accurately, validates tile images are initialized.
c) **Component Registration & Management:** tests adding/removing `GameComponents` updates per-map lists correctly, adding to invalid indices does not crash, ensures components are stored in the correct map.
d) **Map Switching:** confirms `nextMap()` updates map index safely and ensures map transitions do not throw errors and load new stages reliably.
e) **Tile Visibility & Viewport Logic:** validates helper methods that determine if tiles are within the camera view, checks tiles near the player are visible, distant tiles are not, and all visibility checks (`isWorldXLarger`, `isWorldXLess`, etc.) behave consistently.
f) **Walkability & Collision Checks:** Ensures `isWalkable(col,row)` returns true for walkable tiles, false for collidable or out-of-bounds tiles and critical for movement, pathfinding, and gameplay mechanics.

Rationale: Tiles form the backbone of the game world, controlling structure, movement, and interactive elements. Errors in TileManager can cause broken maps, faulty collision, invisible or non-responsive elements, and crashes during map transitions. This test suite ensures that tile data, attributes, component lists, visibility, walkability, and map transitions remain consistent, reliable, and safe under all conditions.

Stats System Tests

Test Class: HealthComponentTest (26 tests)

Features Tested:

- Damage Application: Non-lethal, lethal, exact-lethal, and overkill damage scenarios
- Healing Mechanics: Normal healing, over-healing clamping, healing while dead
- Death State Management: Single death event, no double-death notifications
- Defense Mechanics: Damage reduction calculations
- Edge Cases: Zero damage, negative damage, operations on dead entities
- Constructor Validation: Invalid health/defense values rejected with clear errors

Test Class: StatsTest (23 tests)

Features Tested:

- Attack Data Creation: Basic attacks, ranged attacks, melee attacks
- Critical Hit Mechanics: Random crit calculation, crit damage multiplier
- Stat Getters: Attack, defense, crit chance, crit multiplier retrieval
- Constructor Validation: Negative stats, invalid crit chance/multiplier ranges

Test Class: Player (6 Tests)

Features Tested:

- Initializing the base character correctly and inventory add/remove works as intended
  Making sure the player takes the correct damage, and the player dies if health <= 0
  Testing if crit buff is applied properly and lasts only the intended time

Major Bugs Found:

- The crit buff was not applying properly

Changes:

- Realized a bug where after the buff ended the player's crit chance was reduced to 0, to fix that I initialized and saved the player's crit chance inside the constructor, and fixed how it works inside the update function

Pathfinding Tests

Test Class: PathfinderTest (3 tests)

Features Tested:

- Straight-Line Pathfinding: Direct route when no obstacles
- Obstacle Avoidance: Rerouting around single blocking tile
- Start-Equals-Goal Case: Returns single-tile path when already at destination
- Path Validity: All paths are contiguous (4-way adjacent), walkable, and properly ordered

Test Class: EntityTest (31 tests)

Features Tested:

- Constructor & Initialization: Stats, health, and attack data properly set
- Position Management: Getters for x, y coordinates
- Damageable Interface: takeDamage(), heal(), isAlive(), getCurrentHealth()
- Attacker Interface: attack(), cooldown management, range checking
- Death Handling: onDeath() callback, state transitions

- Health Bar Drawing: UI component rendering (basic existence check)

- *Test Class: EnemyTest (34 tests)*

  Features Tested:

- Constructor Validation: Null checks for dependencies (GamePanel, Pathfinder, Player)
- Pathfinding Integration: Periodic path updates, movement toward player
- Combat Mechanics: Attack when in range, damage application, death drops key
- Death Behavior: onDeath() spawns key, stops movement
- Drawing Effects: Damage flash, health bar, directional sprites
- Direction Handling: getDirection() based on movement vector

  Test class topPanel (4 tests)
- Added a control bar containing Pause, Sound, Zoom, Help, Back, and Exit buttons.
- The zoom buttons are tested to see if the stay within the correct bounds
- For sound, testing to the if it toggles between mute and sounds
- Similar action for Play and pause, tested the clicking and toggling

  Test class dialogueBox (2 tests)
- Tested the visibility of the box on the screen
- Checking if the lines run through all the way before the dialogue closes
- Seeing if the skip button is working when the box is visible
- And seeinf if the skip button is in the right location

II. INTEGRATION TESTS OVERVIEW

Combat-Health Integration

Test: HealthComponentTest.takeDamage_lethalDamage_triggersDeathEvent

> Components Integrated: HealthComponent (health management), CombatManager (event broadcasting), CombatListener (event observer)
> Assertion: Death event should fire exactly once

Enemy-Pathfinder-Player Integration

Test: EnemyTest.update_movesTowardPlayer_usingPathfinder

> Components Integrated: Enemy (AI controller), Pathfinder (A* pathfinding algorithm), Player (target entity), TileManager (world grid)
> Assertion: Enemy should move closer to player after multiple updates

CombatManager Integration

Test: EntityTest.attack_targetInRange_dealsDamage

> Components Integrated: Entity (attacker), Damageable (target interface), CombatManager (attack result notification), Stats (damage calculation)
> Assertion: Target should take damage and attack notification should be sent

Enemy Death-Key Spawn Integration

Test: EnemyTest.onDeath_dropsKey_atEnemyPosition

Components Integrated: Enemy (entity), GamePanel (world manager), ObjectManager (world object spawner
Assertion: Key spawn method should be invoked at enemy's position
Rationale: This tests the game loop's core progression mechanic (kill enemy → collect key → unlock door).

The integration testing of the UI components in regards to the game panel was not tested too much in phase three as other components of the game such as player movement and enemy behavior took a higher priority. so, the percentage coverage on the Game panel is not the highest, though we are planning on increasing our testing coverage.

III. TEST QUALITY MEASURES

We used clear test names as a form of self documentation. We also made sure to test in Isolation creating fresh instances for each test and cleaning up listeners when applicable. We made sure to be aware of edge cases by testing boundary values, invalid inputs, state transitions, and concurrent modifications.

IV. FINDINGS AND IMPROVEMENTS

Bugs Discovered

Bug 1: Double Health Initialization Location: Enemy and Player constructors

Test That Revealed It: constructor_initializesHealthCorrectly() - Was flaky before fix

Bug 2: Duplicate Field Shadowing Location: Enemy and Player classes

Test That Revealed It: constructor_setsGamePanelReference() - Failed before fix due to shadowing

Bug 3: Silent Data Clamping Location: HealthComponent constructor

Test That Drove Fix: constructor_negativeMaxHealth_throwsException()

Code Improvements: Magic Numbers Eliminated, Constructor Validation Added, Removed Unused Code, Added Override Annotations, Cleaner Inheritance, Single Responsibility

Key Learnings:Constructor Validation is Critical Finding: Most bugs were in object initialization phase

V. CODE COVERAGE:

phase2.game.stats: 100% / 88% - Fully tested

phase2.game.combat: 97% / 92% - Nearly complete, minor edge cases in AttackData validation untested

phase2.Tile: 95% / 73% - Well tested, some error handling paths and nextMap untested and updateTraps still have not been handled

phase2.Entity: 69% / 55% - Player input/movement/animations (41%) and KeyItem interactions (15%) untested; Enemy AI well tested (94%)

phase2.UI: 45% / 20% - Main entry point (0%), KeyHandler (3%), GamePanel loop/rendering (31%), and dialogueBox (17%) largely untested due to GUI/interaction dependencies; topPanel display well tested (97%)

Why Certain Components Were Not Tested:

We focused testing efforts on business logic (combat, stats, pathfinding, and map) which are critical to game mechanics and easily testable through unit tests. These components achieved 95-100% coverage because they have clear inputs/outputs and no external dependencies.

VI. CONCLUSION

The testing phase significantly improved code quality, caught multiple initialization bugs, and established a solid regression test suite. The high coverage in combat and stats systems (97-100%) provides confidence in core gameplay mechanics